

Critic-Matched Movie Prediction

Analytic, boosted-tree, and neural designs, each with a raw and a z-score track

Portfolio project — technical report

Rotten Tomatoes critic reviews and Letterboxd member ratings (Kaggle)

Abstract

Two fully parallel projects predict a person’s rating of a film from other raters: critic pseudo-users on Rotten Tomatoes (301 test critics, 43,344 paired episodes) and real members on Letterboxd (300 test members, 41,058 paired episodes). Raters are partitioned 70/15/15 into train/validation/test first, and pseudo-profiles are then amplified within each partition into one cached, seeded training pool per project (§2.5), so no test rater’s ratings ever reach training. Three designs compete on identical held-out episodes in each project: an analytic movie-mean-centered formula (with a second variant restricted to the 10 most-aligned-or-anti-aligned peers), an XGBoost regressor, and a residual neural-network ensemble – all sharing one engineered feature contract that includes, beyond similarity and consensus, real content-based affinity features joined from the gsimonx37/letterboxd Kaggle metadata dump: a user-relative z-scored affinity per genre, a sentence-embedding theme-similarity estimate, per-actor affinity (by rating and by viewing count) with cast-overlap statistics, and per-director affinity (§2.3). On Rotten Tomatoes, full-history RMSE is 0.801 (Design 1), 0.783 (XGBoost), 0.789 (neural net), all beating the 0.843–0.853 flat baselines. On Letterboxd, full-history RMSE is 1.502, 1.417, 1.418 against a 1.654 flat baseline. Every design also trains a parallel z-score track (each rater standardized to their own scale, predicting pure deviation, converted back before scoring) to test whether the gain is calibration or taste-matching; the result is not uniform across datasets or designs, reported as found (§6).

1 Data

1.1 Rotten Tomatoes

The source is `andrezaza/clapper-massive-rotten-tomatoes-movies-and-reviews`. Fractions, letter grades, percentages, and star ratings are parsed and quantized to $\{0, 1, 2, 3, 4, 5\}$, the *standardized raw score* – the prediction target. Critics with at most five parseable scores are excluded, name variants are deduplicated, and repeat critic–movie cells are averaged.

The data retain a diagnostic z-score:

$$z_{c,i} = \frac{r_{c,i} - \mu_c}{\sigma_c}. \quad (1)$$

Here $r_{c,i}$ is critic c ’s standardized raw score for movie i ; μ_c is critic c ’s all-time mean score; and σ_c is critic c ’s all-time standard deviation. The *Tomatometer* is the percentage of reviews labelled Fresh, not a mean 0–5 rating; baseline B2 fits $t_m = aT_m + b$ (a linear calibration of movie m ’s Tomatometer percentage onto the raw score scale).

Movie facets. RT’s own `genre` column is a comma-separated multi-genre string, parsed in full (not just the first entry). This is joined to the gsimonx37/letterboxd Kaggle metadata dump (`genres`, `themes`, `actors`, `crew`, `languages` – `studios/countries` are downloaded but not used,

Data funnel	Count
Raw review rows (start)	1,444,963
Distinct critics with a parseable score	8,623
After the low-volume critic floor (> 5 scored reviews)	4,393 critics, 1,262,747 rows
Parsed, standardized scored reviews used for modelling	992,954
Distinct movies with any scored review	58,736
Neighbour pool / pseudo-users (≥ 10 distinct movies)	2,894 critics
Movies in the final app catalog (most-reviewed)	1,000
Critics appearing in that catalog	3,387

Table 1: From 1.4M raw rows to the modelling pool and the app catalog.

and posters are excluded entirely) by normalized (title, year), with title-only fallback when a year is missing (RT’s own release-date field is populated for only 30,512 of 142,052 movies). Five facets survive into the feature contract: genre, theme, language, director, and actor (the 20 most-billed actors per film – see §2.3); decade and studio/country are excluded, decade because **year** already carries that signal and studio/country because they cost more columns than they were worth. The join matches **69.2%** of RT’s full movie catalog and **99.3%** of the 1,000-film app catalog (popular films skew toward better metadata coverage); unmatched films fall back to RT’s own genre string and empty facets elsewhere, reported honestly rather than assumed complete. Rating-weighted across all scored reviews (i.e. weighting each film’s coverage by how often it was actually rated, not by raw film count), **67.2%** of reviewed films have at least one theme and **82.3%** have at least one credited actor; among films with only 3 or more raters, theme coverage drops to **36.2%** – the theme affinity feature (§2.3) is NaN whenever neither the target nor any seen film has themes.

1.2 Letterboxd

To test whether real, dense per-user rating histories beat the sparse critic pseudo-users above, a fully parallel second project was built on Letterboxd community ratings (Kaggle, ~11.08M ratings), isolated in its own directory tree (`src/letterboxd/`, `data/letterboxd/`, `results/letterboxd/`) sharing no data, models, or code with Rotten Tomatoes. The export yields 7,420 members with at least five rated films across 286,069 films – 11,078,045 ratings total. Ratings are native on a 1–10 scale (no score parsing, no Tomatometer feature).

The same `gsimonx37` join is repeated independently (title, year; LB’s own `movie_id` slug already encodes the year, so the join is more reliable here) and matches **94.3%** of LB’s full catalog and **99.8%** of the 1,000-film app catalog. LB’s own `genres` column (a JSON list, already multi-genre) is the fallback when `gsimonx37` misses. Coverage is denser here than on RT (LB’s join is more reliable, and its raters skew toward well-catalogued films): rating-weighted, **78.8%** of rated films have at least one theme and **96.4%** have at least one credited actor; among films with only 3 or more raters, theme coverage is **15.4%**.

2 Equations and concepts

2.1 Similarity and magnitude

Let $O_{u,c}$ be movies rated by pseudo-user/visitor u and peer (critic/member) c among the sampled seen movies. Pearson correlation is

$$\rho_{u,c} = \frac{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})(r_{c,i} - \bar{r}_{c,O})}{\sqrt{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})^2} \sqrt{\sum_{i \in O_{u,c}} (r_{c,i} - \bar{r}_{c,O})^2}}. \quad (2)$$

Small overlaps are shrunk toward zero:

$$s_{u,c} = \rho_{u,c} \frac{\min(n_{u,c}, k)}{k}, \quad n_{u,c} = |O_{u,c}|, \quad k = 8. \quad (3)$$

The rating-magnitude multiplier (a least-squares through-origin mapping from peer to user ratings) is

$$g_{u,c} = \frac{\sum_{i \in O_{u,c}} r_{u,i} r_{c,i}}{\sum_{i \in O_{u,c}} r_{c,i}^2}. \quad (4)$$

Pearson correlation is affine-invariant, so $\rho_{u,c}$ (and $s_{u,c}$) is *identical* whether computed in raw units or z-units; only $g_{u,c}$ and the target consensus differ between the raw and z-score tracks (§2.4).

2.2 Movie-mean-centered prediction and the top-|sim| variant

For target movie m , let C_m be other peers who rated it, with movie mean $\bar{r}_m = \frac{1}{|C_m|} \sum_{c \in C_m} r_{c,m}$. The full-neighbourhood prediction is

$$\hat{r}_{u,m} = \frac{\sum_{c \in C_m} (|s_{u,c}| \bar{r}_m + s_{u,c}(r_{c,m} - \bar{r}_m)) g_{u,c}}{\sum_{c \in C_m} |s_{u,c}|}, \quad (5)$$

clipped to the dataset’s rating range; a zero denominator falls back to $\bar{r}_m$.

A second, new variant restricts C_m to $C_m^{(k)} \subseteq C_m$, the $k = 10$ peers with the largest $|s_{u,c}|$ – both strongly aligned ($s_{u,c} \gg 0$) and strongly anti-aligned ($s_{u,c} \ll 0$) peers, not just the most positively-aligned like baseline B4:

$$C_m^{(k)} = \arg \text{top-k}_{c \in C_m} |s_{u,c}|, \quad \hat{r}_{u,m}^{(k)} = \frac{\sum_{c \in C_m^{(k)}} (|s_{u,c}| \bar{r}_m^{(k)} + s_{u,c}(r_{c,m} - \bar{r}_m^{(k)})) g_{u,c}}{\sum_{c \in C_m^{(k)}} |s_{u,c}|}, \quad (6)$$

where $\bar{r}_m^{(k)}$ is the mean over the restricted set $C_m^{(k)}$. Equation 6 is otherwise identical to Equation 5 – same shrinkage, same magnitude term – just over a smaller, alignment-selected neighbourhood. The original full-neighbourhood formula is unchanged and remains the default; the top-|sim| variant is purely additive, computed identically for both datasets.

2.3 Feature engineering

Every trained design (Design 2 and 3) sees the same fixed-width row, built once here and consumed identically by both. It has four parts: a peer-similarity summary, target-side scalars, user-relative content affinities (the focus of this section), and target-side metadata.

Decile summary (30 columns). Each peer’s target-movie deviation is its leave-one-out all-time mean subtracted from its target score. Peers are ranked by $s_{u,c}$ and their $s_{u,c} \times$ deviation products are summarized (mean, count, standard deviation) in ten similarity-ranked deciles – identical construction for both datasets and both raw/z tracks.

Target-side scalars (7/6 columns). Seen count, mean/max peer overlap, peer count, the target’s rating dispersion across all its raters, mapped Tomatometer (RT only), and the user’s own mean rating. No single-genre ordinal is included: the 40-column genre affinity block below covers every genre a film has (not just its first), so a separate `genre_id` column would be redundant.

Content affinities (77 columns). Built from the `gsimonx37` movie-facet join (§1.1, §1.2). A design note on why these are per-value blocks rather than a coarser “mean deviation over any shared facet value” aggregate: the aggregate form cannot distinguish a film matching the target on two facet values from one matching on one, and its zero-count case is ambiguous between “no shared value” and “target has no data for this facet.” Each block instead computes, per user-relative quantity, a genuine estimate of how the user has rated similar content before, always paired with a confidence count that doubles as its own missingness indicator (a NaN average’s paired count is always exactly 0 or, for genre, at most 1 – see below). Let μ_u, σ_u be the mean and standard deviation of the user’s own seen-set ratings for this episode (raw track: the actual seen ratings; z-track: the already-standardized per-episode z-values, so $\mu_u \approx 0, \sigma_u \approx 1$ there by construction); when $\sigma_u < 10^{-9}$ (a seen set with no rating variance, most often $n = 1$), σ_u falls back to σ_{global} , the standard deviation of every rating in the dataset.

Genre (40 columns). Let G be the canonical genre vocabulary – the 19 named `gsimonx37` genres plus one “__other__” slot, so $|G| = 20$ regardless of catalog. For genre $g \in G$ and seen set S with per-film ratings r_i :

$$z_g(u, m) = \frac{\bar{r}_{H_g} - \mu_u}{\sigma_u}, \quad H_g = \{i \in S : g \in F_{\text{genre}}(i)\}, \quad (7)$$

undefined (NaN) when $H_g = \emptyset$; $\bar{r}_{H_g}$ is the mean seen rating over H_g . The paired count *includes the target film itself*:

$$\text{cnt}_g(u, m) = \log(1 + |H_g| + \mathbb{I}[g \in F_{\text{genre}}(m)]). \quad (8)$$

This single count column replaces a separate target-side multi-hot entirely: $\text{cnt}_g = 0$ means the target isn’t genre g ; $\text{cnt}_g = \log 1$ (i.e. the +1 alone) means the target is genre g but the user has seen none; $\text{cnt}_g = \log(1 + n)$ for $n \geq 1$ seen films means both. It is target-side information only ($\mathbb{I}[g \in F_{\text{genre}}(m)]$ needs no rating), so it adds no leakage.

Theme (3 columns). The 109 `gsimonx37` theme phrases are short descriptive English sentences (e.g. “*Gothic and eerie haunting horror*,” “*Terrifying, haunted, and supernatural horror*”) – similar in meaning but sharing no facet string, so a set-overlap affinity (as genre/director/actor use) would miss the relationship entirely. Each phrase is instead encoded once, offline, with a sentence-transformer (`all-MiniLM-L6-v2`, L2-normalized), giving a 109×109 cosine similarity matrix computed a single time and cached (seconds on CPU); the two horror phrases above cosine at 0.79, against 0.21–0.27 to an unrelated theme – concrete evidence the embedding captures what string matching cannot. For target themes T_m and a seen film i with themes T_i , define $w(t, i) = \max_{\tau \in T_i} \cos(e_t, e_\tau)$ for $t \in T_m$ (0 if $T_i = \emptyset$). The theme feature is a similarity-weighted average of the *raw* seen rating (not a deviation – it is a kernel-smoothed rating estimate over the theme graph, meant to be directly comparable to the prediction target):

$$\text{avg}_\theta(u, m) = \frac{\sum_{t \in T_m} \sum_{i \in S} w(t, i) r_i}{\sum_{t \in T_m} \sum_{i \in S} w(t, i)}, \quad (9)$$

undefined when $T_m = \emptyset$ or the denominator is 0. (This is algebraically the mass-weighted combination of each theme’s own weighted average – the per-theme intermediate cancels.) The mass itself, $\log(1 + \sum_{t,i} w(t, i))$, is the confidence companion and the exact mask for Equation 9 (zero iff the average is NaN); a third column counts (log1p) seen films with $\max_{t \in T_m} w(t, i) > 0.8$, i.e. strongly on-theme.

Actor (30 columns). Cast depth is the 20 most-billed actors per film – deep enough that cast-overlap statistics carry signal (the gsimonx37 dump’s median cast is 6, 90th percentile 20; a 2–3-actor cast would make overlap nearly always empty). Three top-5 rankings of the target’s cast against the user’s seen history: (i) by the user’s mean rating for that actor, tie-broken by count; (ii) by the user’s viewing count for that actor, tie-broken by mean rating; (iii) seen films ranked by shared-cast count with the target, keeping each film’s own raw rating (a single observation, not averaged – `user_mean` already carries the user’s general level). Each ranking contributes a z-score/rating and a log1p count per rank; unused ranks (fewer than 5 matching actors/films) are padded with NaN (z-scores and ratings) and 0 (counts).

Director (2 columns). One z-score and one log1p(count), identical construction to genre/actor, over seen films sharing any director with the target (no target-inclusive count here – directors are near-unique per film, so the genre trick’s leakage-free target bit adds little).

Numeric tail (4/4 columns). $\log(1 + \text{runtime})$, the gsimonx37 dataset’s own average site rating for the film, and $\log(1 + n_{\text{themes}})$, $\log(1 + n_{\text{languages}})$.

Missing-value convention. XGBoost splits on NaN natively (missing-value direction), so it needs no imputation. The neural network cannot use NaN directly, so every NaN-able column is imputed with a fixed sentinel before standardization: rating-scale averages (theme average, cast-overlap ratings) get -1 (impossible on both the 0–5 and 1–10 scales); z-scores get 0 (their own neutral point – “no evidence” and “exactly average” are deliberately not distinguished further, since the paired count already grades how much evidence there is). No separate missingness mask is used: every NaN-able column already has a count companion that *is* its mask, and the count does more than flag absence – it grades how weak the evidence is, so zero, one, and two prior observations fall in one smoothly learnable region rather than requiring a hard binary split.

In total the feature row has **116** columns on Rotten Tomatoes (with Tomatometer) and **115** on Letterboxd.

2.4 The z-score track

Every design also trains a parallel z-score variant to test directly whether its gain over the baselines is level-calibration or genuine taste-matching. Each rater is standardized to their own scale using **two different conventions**, deliberately:

- **The user** (the fake pseudo-user offline, or the real visitor in the app) is standardized by the mean/std of *only the ratings sampled as seen for this episode* – never a critic/member’s all-time stats – since a real visitor only ever has their own seen films to standardize against: $z_{u,i} = (r_{u,i} - \mu_u) / \sigma_u$ where μ_u, σ_u come from that episode’s seen set.
- **Peers** (critics/members) are standardized by their *all-time* mean/std – Equation 1 for RT, the analogous all-time computation for LB.

The model predicts pure deviation $\hat{z}$ in this mixed z-space, then converts back to the raw scale before scoring:

$$\hat{r} = \text{clip}(\mu_u + \sigma_u \hat{z}, \text{range}). \quad (10)$$

$\text{user_mean} \approx 0$ by construction on this track (the level was removed), and the content-affinity blocks (§2.3) use $z_{u,i}$ directly as the seen-set "rating" they average over – already ≈ 0 -mean, ≈ 1 -std, so those blocks are already scale-free on this track without any extra transformation. RMSE is always reported after Equation 10, so raw-track and z-track numbers are directly comparable (§6.3). An episode is skipped on the z-track only if its seen ratings have zero variance ($\sigma_u = 0$, can't standardize).

2.5 Sampling protocol and the training pool

Split first, then amplify. Every rater (critic on Rotten Tomatoes, member on Letterboxd) is assigned to exactly one of train/validation/test by a deterministic seeded shuffle – **70 / 15 / 15** in both projects. On Rotten Tomatoes the 2,894 pseudo-user critics split 2,026 / 434 / 434; on Letterboxd the 7,409 eligible members (≥ 6 ratings) split 5,186 / 1,111 / 1,112. All pseudo-profile sampling happens *within* a partition, so every profile derived from one rater stays in that rater's split: no rating belonging to a test rater is ever visible at training time.

Pseudo-profile amplification. A single rater with a long history supports many distinct training examples: an episode fixes one held-out target film and samples n of the rater's remaining films as the "seen" set. Training and validation rows draw, per rater, several independent random profiles at every seen-count in the evaluation grid $n \in \{3, 5, 10, 20, 50, \text{all}\}$ (32 train / 8 validation profiles per n on Rotten Tomatoes; 4 / 2 on Letterboxd, whose raters are $\sim 2.5\times$ more numerous and far denser). Every rater draws from its own RNG stream, seeded as `default_rng([seed, rater_position])` – independent of processing order and worker count, so the pool is bit-for-bit reproducible whether generated serially or across processes.

One cached pool, shared by both trained designs. The full pool – training, validation, and paired test rows, in both the raw and z-score tracks – is generated once by a dedicated pipeline stage (`build_rows`, `make rt-rows / lb-rows`) and cached to parquet with a manifest recording the seed and a hash of the feature contract. XGBoost and the neural network both load this cache, so the two trained designs fit on byte-identical rows and are scored on byte-identical test episodes; a stale cache (changed contract or seed) is refused rather than silently regenerated. The pool sizes: 358,080 training rows per track on Rotten Tomatoes and $\sim 125,000$ on Letterboxd (every train member contributes, capped only by each member's own history length). A raw-track row and its z-track twin are built from the *same* sampled episode, and an episode is dropped from both tracks if either row is unavailable, keeping the tracks 1:1 comparable.

Paired, nested test episodes. Test raters are evaluated on the shared paired protocol (§3–5): per eligible test rater (more than 50 rated films), 8 fixed targets $\times$ 3 popularity-weighted seen-order redraws, with the seen set at count n a nested prefix of the same order – so every seen-count, every design, and every baseline is scored on an identical (rater, target, draw) set.

2.6 Metrics

Primary error is

$$\text{RMSE} = \sqrt{\frac{\sum_{j=1}^N (\hat{r}_j - y_j)^2}{N}}, \quad (11)$$

where j indexes an episode. Two secondary metrics (§8) check whether the gain is calibration or reordering: within-user Spearman rank correlation, and precision@10 (overlap between a user’s predicted and true top-10 held-out films).

3 Design 1: analytic formula

3.1 Raw

Design 1 is Equation 5 (full neighbourhood) and its new top-|sim| variant, Equation 6, computed identically on both datasets from the same similarity/magnitude substrate (§2). No trained parameters beyond the validation-selected shrinkage constant $k = 8$ (Equation 3) and the neighbourhood size $k = 10$ for the top-|sim| variant. B4 (the existing baseline) selects the k most *positively*-aligned peers and averages unweighted; the new Design-1 variant selects by $|s_{u,c}|$ (so it can include strongly anti-aligned peers) and applies the full weighted formula, not a plain average.

3.2 Z-score

Both the full-neighbourhood and top-|sim| formulas are recomputed against the peer pool’s all-time z-values (Equation 1) instead of raw scores, with $g_{u,c}$ and the target consensus in z-units; the shrunk similarity $s_{u,c}$ is unchanged (affine invariance, §2). The z-space prediction is converted back via Equation 10 using *this episode’s* seen-set μ_u, σ_u before clipping and scoring.

4 Design 2: XGBoost

4.1 Raw

An XGBoost regressor (scikit-learn estimator API, `XGBRegressor`) over the full 116/115-column feature row (§2.3): the 30 decile columns, seen-count/overlap/dispersion/reviewer-count, mapped Tomatometer (RT only), the 40-column genre block, the 3-column theme block, the 30-column actor block, the 2-column director block, and the 4-column numeric tail. Missing Tomatometer and every NaN-able affinity column are handled natively by XGBoost’s missing-value split direction. Trained with early stopping on validation RMSE. Both tracks are trained as regressors rather than multiclass classifiers, even though the raw targets fall on a discrete grid (0–5 on Rotten Tomatoes, 1–10 on Letterboxd): the z-score track’s target is continuous by construction, and the reported metric is RMSE on the original scale, which rewards ordered, interpolating predictions – a multiclass objective would treat every misclassification as equally wrong regardless of how far off it is.

4.2 Z-score

A second XGBoost model is trained on the identical feature contract, but every peer-side quantity (deciles, facet peer-mean, tomatometer stays raw as an external feature) is built from the z-Split, and the target is $\hat{z}$; predictions are converted back via Equation 10 before scoring, using the same episode’s μ_u, σ_u as the raw track (so raw and z rows are drawn from byte-identical sampled episodes).

5 Design 3: neural network

5.1 Raw

A residual MLP ensemble in PyTorch (Apple GPU/MPS), architecturally identical for both datasets: the full 116/115 numeric columns – no separate genre embedding, since the 40-column genre affinity block already gives the network richer per-genre information (every genre a film has, not just a single first-genre id) than an embedding lookup would – log1p-compressed where they are counts, NaN-imputed with the fixed per-column sentinel of §2.3 (not the column mean, so a missing affinity always lands at the same point regardless of what a given split’s training rows happened to average), and standardized on train statistics, feed a projection directly into six pre-activation residual blocks of width 512 (3,238,353 weights per ensemble member; batch-norm, GELU, dropout), then a linear head. Trained with AdamW, ReduceLROnPlateau, and early stopping; three independently seeded networks are averaged into an ensemble. Both trained models are verified leakage-free: peer means are leave-one-out, u /the visitor is excluded everywhere, and every content-affinity block (§2.3) is built only from seen films.

5.2 Z-score

A second ensemble, identical architecture, trained on the z-Split’s rows with target $\hat{z}$; the three z-track members are seeded from a distinct offset so they are not literal duplicates of the raw-track members. Predictions convert back via Equation 10 before scoring, on the same paired test episodes as every other method.

6 Results

6.1 Rotten Tomatoes

Design 1 improves over B3 by 5.0% at full history; the trained models win everywhere: XGBoost overall/full-history RMSE 0.807/0.783, the neural net 0.809/0.789. The top-|sim| variant of Design 1 trails the full-neighbourhood formula slightly at every seen-count (0.811 vs. 0.801 full-history) – restricting to 10 alignment-selected peers discards useful information the full weighted sum still captures, an honest negative result for that variant. Across low/mid/high critic-disagreement terciles, full-history gain over B2 is +8.3/ +7.5/ +3.9% for Design 1, +10.2/ +8.2/ +7.3% for XGBoost, +11.2/ +7.4/ +5.6% for the neural net.

Magnitude matching supplies the entire improvement over the consensus; the signed similarity term adds nothing on top – as with the trained models (§6.1), correlation-based taste matching contributes little.

Seen ratings n	3	5	10	20	50	all
B1 global mean	1.072	1.072	1.072	1.072	1.072	1.072
B2 Tomatometer $\rightarrow$ score	0.853	0.853	0.853	0.853	0.853	0.853
B3 reviewer mean	0.843	0.843	0.843	0.843	0.843	0.843
B4 top-10 similar mean	0.917	0.906	0.893	0.880	0.872	0.858
Design 1 (full)	0.971	0.923	0.874	0.844	0.812	0.801
Design 1 (top- sim)	0.993	0.948	0.902	0.864	0.835	0.811
Design 2 XGBoost	0.830	0.822	0.812	0.804	0.793	0.783
Design 3 neural net	0.831	0.823	0.815	0.805	0.794	0.789

Table 2: RMSE on the 43,344 paired, nested episodes (301 eligible test critics with more than 50 rated films, out of the 434-critic test partition; 2,894 pseudo-users total). The trained designs are scored on the 42,393-episode subset where the z-track row also exists, keeping the two tracks 1:1 comparable. Baselines are flat because they do not depend on the seen set.

Formula component (full-history episodes)	RMSE
Reviewer mean / movie consensus	0.850
Magnitude-scaled movie consensus	0.833
Movie-centered signed similarity ($g_{u,c} = 1$)	0.867
Full Design 1 formula	0.850

Table 3: Formula attribution: magnitude scaling supplies the whole gain over consensus.

Both trained models lean first on the movie’s Tomatometer/consensus, then the user’s mean; scaling the network wider, deeper, and on more data moved the loss by nothing in earlier iterations of this experiment, and ablations that repackaged the same information (a coarser facet-overlap scheme in place of the per-genre/theme/actor/director affinity blocks of §2.3) shifted full-history RMSE by well under 0.01 – on this engineered tabular feature set the ceiling is set by the information in the features, not model capacity or how that information is packaged, and content-level personalization remains a small effect on top of calibration, not a replacement for it. The affinity features are feature-importance-visible, however: the top-ranked XGBoost columns include individual genre and actor affinities rather than only consensus and user-level scalars, giving an interpretable picture of what the model actually leans on (`results/rotten_tomatoes/tables/design2_feature_importance.csv`).

6.2 Letterboxd

Seen ratings n	3	5	10	20	50	all
B1 global mean	2.104	2.104	2.104	2.104	2.104	2.104
B3 mean of all members	1.654	1.654	1.654	1.654	1.654	1.654
B4 top-10 similar mean	1.710	1.716	1.680	1.671	1.654	1.614
Design 1 (full)	1.715	1.648	1.575	1.545	1.527	1.502
Design 1 (top- sim)	1.806	1.723	1.633	1.591	1.553	1.501
Design 2 XGBoost	1.611	1.586	1.548	1.521	1.480	1.417
Design 3 neural net	1.610	1.579	1.541	1.518	1.494	1.418

Table 4: Letterboxd RMSE (1–10 scale) on the same paired, nested seen-history protocol: 300 deterministic test members, 8 fixed targets each, 3 seen-order redraws (41,058 episodes).



Figure 1: Error versus sampled seen history (Rotten Tomatoes). Both trained models beat every baseline at every seen-count.

Design 1’s top-|sim| variant lands in a statistical dead heat with the full formula at full history here (1.501 vs. 1.502) while trailing it at every finite seen-count – a more favourable showing than on Rotten Tomatoes, where the full formula stays ahead throughout, but still no positive case for restricting the neighbourhood. The trained models are effectively tied for best full-history method on Letterboxd (XGBoost 1.417, neural net 1.418), consistent with §6.1’s finding that beyond calibration and consensus the two very different model families extract the same information from this feature contract.

Honest cross-dataset finding. Normalizing by rating range ($\text{RMSE} / (\text{max} - \text{min})$) puts the two 0–5 and 1–10 scales on the same footing. Rotten Tomatoes’ normalized RMSE is 0.160/0.162/0.157/0.158 (Design 1 full / Design 1 top-|sim| / XGBoost / neural net); Letterboxd’s is 0.167/0.167/0.157/0.158. The trained models land at essentially *identical* normalized error on both datasets, and the analytic formula normalizes marginally better on Rotten Tomatoes despite its far sparser critic pseudo-user profiles – Letterboxd’s dense real member histories did not translate into a lower normalized error anywhere, even with the content-affinity features (§2.3) available to both. Full numbers are in `results/letterboxd/cross_dataset_comparison.csv`.

6.3 Isolating scale: the z-score track

The result is **not uniform**, reported as found rather than presupposed. On Rotten Tomatoes the z-track trails raw at every design; the full-neighbourhood Design 1 loses the most (+0.147) since it has no learned capacity to reallocate, while the trained models lose far less (+0.003 to +0.010). On Letterboxd, the trained models (Design 2/3) come out *flat-to-slightly-better* in z-space, and the top-|sim| analytic variant is indifferent (−0.001) – only the full-neighbourhood formula still

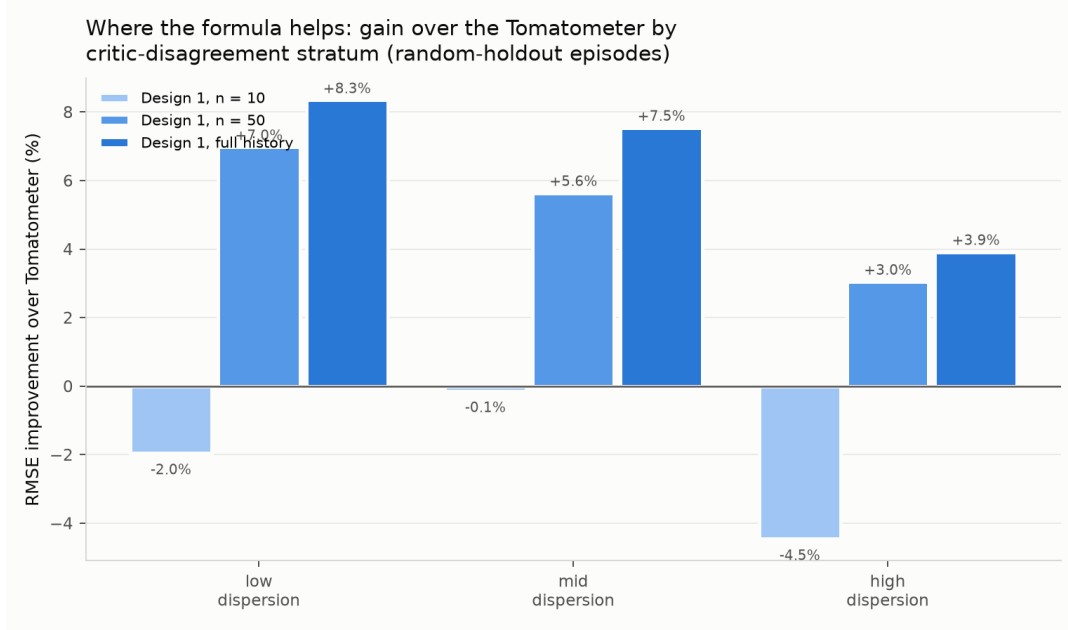


Figure 2: Design 1 gain over B2 by target-movie critic disagreement.

Design	raw RMSE	z RMSE (converted back)	z - raw
<i>Rotten Tomatoes (0-5)</i>			
Design 1 (full)	0.801	0.948	+0.147
Design 1 (top- sim)	0.811	0.844	+0.032
Design 2 XGBoost	0.783	0.792	+0.010
Design 3 neural net	0.789	0.793	+0.003
<i>Letterboard (1-10)</i>			
Design 1 (full)	1.502	1.607	+0.105
Design 1 (top- sim)	1.501	1.501	-0.001
Design 2 XGBoost	1.417	1.412	-0.005
Design 3 neural net	1.418	1.406	-0.013

Table 5: Full-history RMSE, raw vs. z-score track, both reported on the raw scale.

loses meaningfully (+0.105). A plausible reading: restricting to the top-|sim| peers, or handing a model spare learned capacity, both let the deviation-only signal be used more directly once the rater’s own level is removed; the full-neighbourhood analytic formula never had that capacity to reallocate, so it only loses the calibration information without regaining anything. Either way, the effect is small relative to raw calibration’s gain over the flat baselines (Table 2, 4) – calibration, not taste-matching, remains the dominant effect in this data.

7 Similar-film suggestions: does rating them help?

Each row of the app’s “films to predict” table offers a “movies like this one” dropdown: rate one of the target film’s nearest content neighbours and the prediction should, in principle, improve. Neighbours are precomputed offline with K-means: a standardized feature vector per catalog film (the genre multi-hot from §2.3, a catalog-scoped top- k multi-hot over director/actor/theme/language,

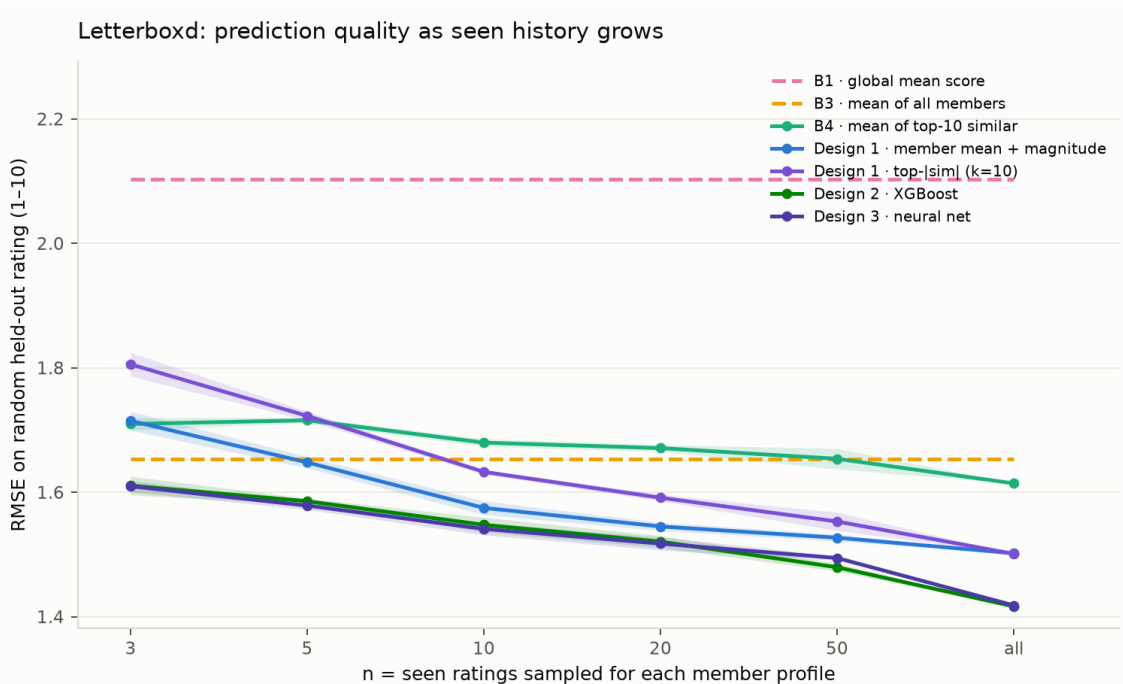


Figure 3: Letterboxd: error versus sampled seen history, styled identically to Figure 1.

and numeric runtime/external-rating/year/consensus), clustered into $\max(2, \lfloor n/25 \rfloor)$ groups, with each film’s `k_neighbors` nearest neighbours (Euclidean distance) taken from within its own cluster (topped up from the globally nearest remaining films if the cluster is too small). `k_neighbors` is the dropdown’s list length, identical on both datasets. Studio/country/decade are excluded from this vector for the same reasons they are excluded from the model feature contract (§2.3).

Does it work? 6,000 paired (rater, $n=10$ seen, target) episodes were drawn from each project’s own 1,000-film catalog (Design 1, leave-one-out, identical protocol to §3) and bucketed by how many of the target’s `k_neighbors` nearest neighbours were already in that episode’s seen set – the same 6,000 predictions are reused for every `k_neighbors` value, so only the bucketing (which films count as “similar”) differs.

On **Rotten Tomatoes** there is *no clean beneficial signal*. At $k=20$ the per-bucket RMSE by similar-seen count is 0.796 (0 seen, $n=4,570$), 0.772 (1, $n=1,196$), 0.869 (2, $n=193$), 0.924 (3, $n=35$), 0.830 (4, $n=6$) – essentially flat from 0 to 1 seen, then *rising* as more similar films are rated, the opposite of the hoped-for direction, though the higher-count buckets are thin enough (35 and 6 episodes) that little should be read into the exact shape. Pooling every bucket with at least three similar films seen (dropping buckets under five episodes) gives one summary per k : 0.910 at $k=20$ ($n=41$) and 0.900 at $k=30$ ($n=89$), both *above* the 0-seen baseline for their own k (0.796 and 0.799) – rating more of a film’s nearest neighbours did not measurably improve its prediction in this sample.

On **Letterboxd**, the sweep is noisier still and shows no signal either: at $k=20$, RMSE by similar-seen count is 1.589 (0, $n=4,651$), 1.555 (1, $n=1,183$), 1.565 (2, $n=143$), 1.682 (3, $n=16$), 2.006 (4, $n=4$) – flat within noise up to 2 seen, then rising on buckets thin enough (16 and 4 episodes) to be unreliable. The pooled ≥ 3 -similar-seen summary is 1.682 at $k=20$ ($n=16$, below the 5-episode floor for a trustworthy weighted average so reported as a single bucket) and 1.588 at $k=30$ ($n=53$) – both in-

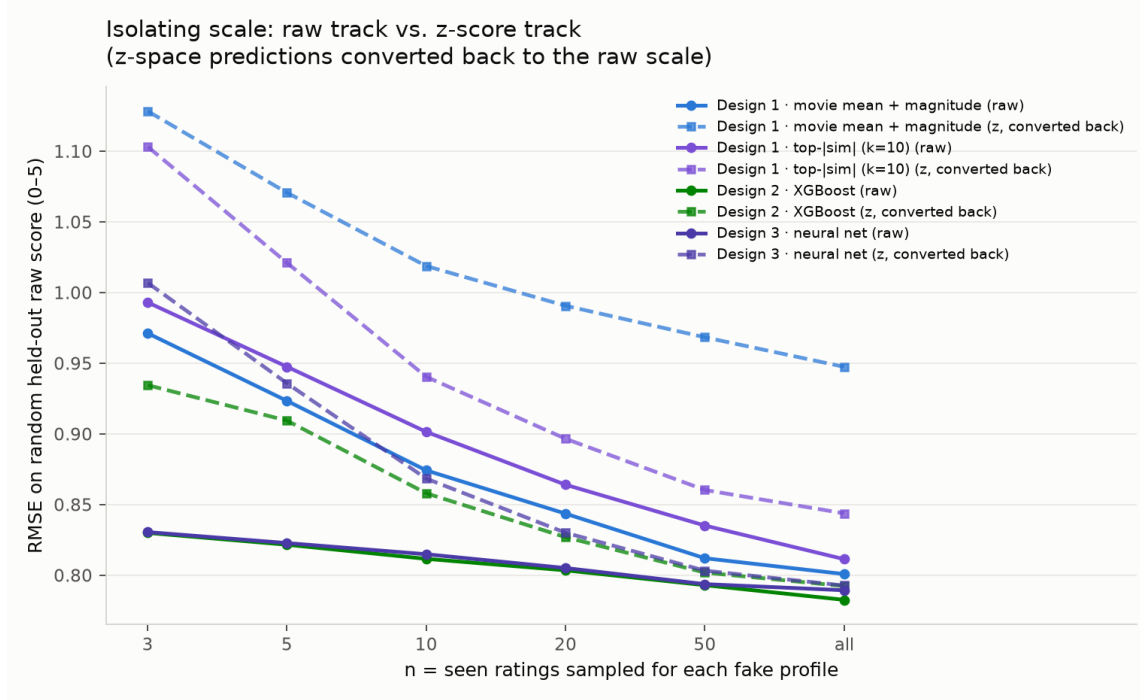


Figure 4: Rotten Tomatoes: raw (solid) vs. z-score (dashed) RMSE by seen count, per design.

distinguishable from, or slightly above, the 0-seen baseline (1.589 and 1.598). **Reported honestly:** on both datasets, rating a target film’s nearest content neighbours first does not measurably sharpen the prediction in this sample – a negative result for the “movies like this one” hypothesis, not a confirmation. `k_neighbors= 20` is kept as the dropdown length on both projects regardless, since it is a UI choice (how many suggestions to show) independent of this null predictive-value finding. Full per-bucket numbers are in `results/{rotten_tomatoes,letterboxd}/similar_k_sweep.csv` (`rotten_tomatoes.similar_k_sweep` / `letterboxd.similar_k_sweep`).

8 Application and limitations

The website (Table 1: 1,000 catalog movies, 3,387 critics / 7,416 members) is organized in three sections, with a sort control (alphabetical, year, most-reviewed) and a **genre filter** (a dropdown built from each film’s `gsimonx37` genre list, §1.1–1.2) over the search lists. (1) *Films you have seen*: a search box adds a film to a table where each row carries a rating widget (five-star on Rotten Tomatoes, a 1–10 stepper on Letterboxd). (2) *Films to predict*: the same searchable table, scoring optional; each row also has a “movies like this one” dropdown (§7) – green if you have already rated one of its 20 nearest content neighbours, red otherwise. (3) *Predictions*: for every target film the app shows all four model variants (Design 1 full, Design 1 top-|sim|, XGBoost, neural net) side by side, each with its raw and z-score prediction, plus the movie’s consensus mean and (Rotten Tomatoes only) Tomatometer. When the visitor scores some target films, the app reports the mean squared error of every variant against the visitor’s own score and marks the closest, answering “which method predicts me best?”. All eight model variants (four designs × two tracks) run entirely client-side, ported from the Python training code to hand-written TypeScript (an XGBoost tree-walker, a neural-net forward pass, and the content-affinity feature computation of §2.3, verified to

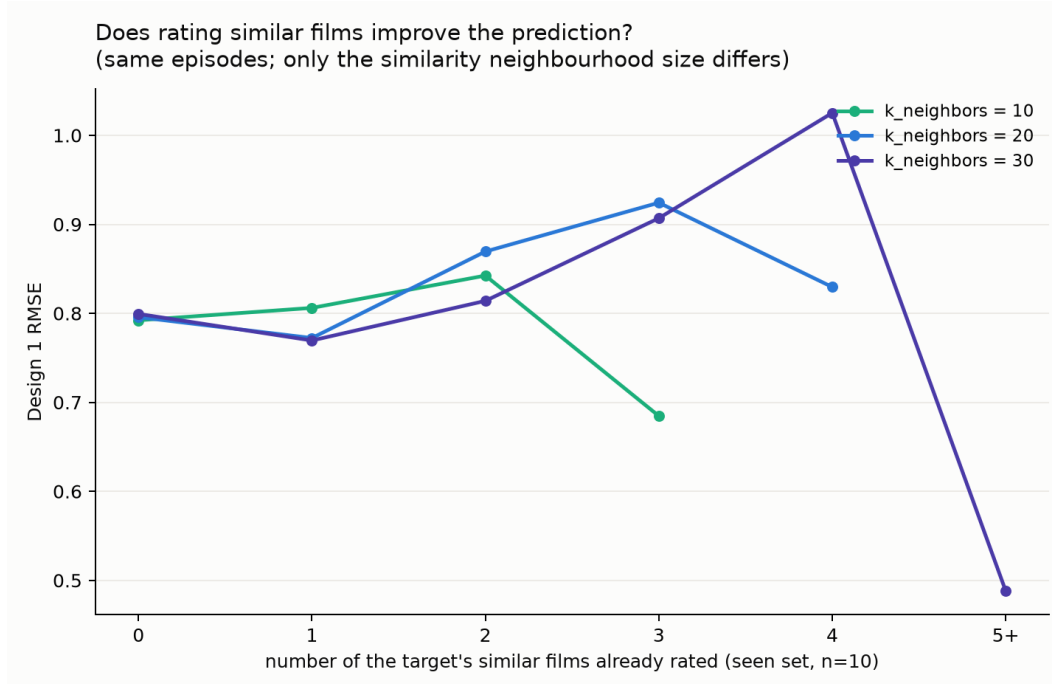


Figure 5: Rotten Tomatoes: Design 1 RMSE by how many of a target’s `k_neighbors` nearest neighbours are already rated, for `k_neighbors` $\in \{10, 20, 30\}$.

match Python to within numerical tie-break tolerance – see `web/scripts/validate*.ts`). The theme block’s sentence-transformer runs *only offline*, once, over the 109 known theme phrases; the browser receives just the resulting 109×109 cosine-similarity matrix (a ~ 48 KB JSON file) and does a dot-product lookup, never transformer inference.

Two secondary metrics (`rotten_tomatoes.analyze`, `secondary_metrics.csv`) check that the gain is calibration rather than reordering: within-user Spearman rank correlation between predictions and held-out ratings, and `precision@10`. Design 1’s within-user Spearman is 0.587 versus 0.578 for the reviewer mean (B3), and `precision@10` is 0.287 versus 0.284 – ranking is barely changed, consistent with the attribution finding (Table 3) that the win is calibration, not reordering a user’s films.

Item holdout is not temporal forecasting. The `gsimonx37` join is fuzzy (normalized title + year) and RT’s own release-date coverage is sparse, so its match rate (69.2% full catalog) is lower than Letterboxd’s (94.3%) – reported honestly rather than assumed complete; unmatched films fall back to each project’s own (sparser) genre parsing. The `top-|sim|` analytic variant underperforms the full formula on both datasets – a negative result, not tuned away. The magnitude multiplier is an unregularized through-origin slope. Scores are quantized to six levels on Rotten Tomatoes; pseudo-users there are professional critics rather than casual users.

Repository layout

Both projects use one flat module layout, file-for-file: `src/rotten_tomatoes/` holds `analytic.py` (the formula, `top-k` baseline, and the `top-|sim|` variant), `train_analytic.py` (the `k`-sweep and paired test), `build_rows.py` (the cached training pool of §2.5), `train_xgboost.py`, `train_neural.py`

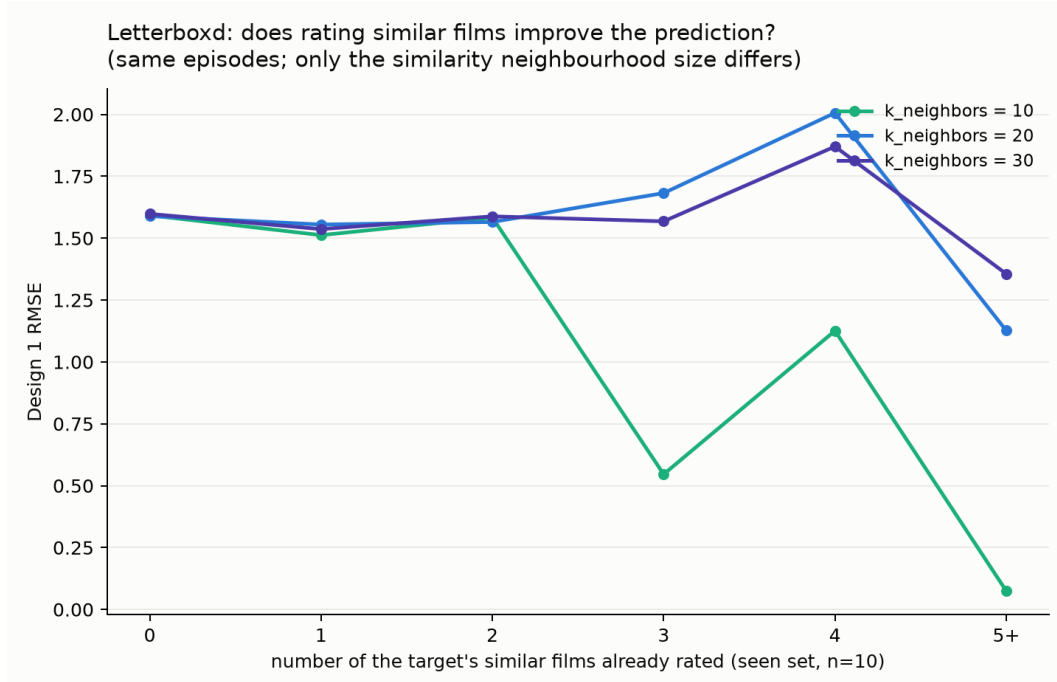


Figure 6: Letterboxd: the same sweep, styled identically to Figure 5.

(with `network.py`), `analyze.py` (the cross-design comparison), `plots.py` (shared RMSE-by- n plotting plus each trainer’s per-run scratch plot), `attribution.py`, `similar_k_sweep.py` (the neighbour-list tuning of §7), the three `predict_*.py` app-inference modules, and `preprocess.py/audit.py` for ingest. The pseudo-user substrate (`pseudo_users.py`), the shared feature contract (`features.py`), and the gsimonx37 movie-facet join (`movie_features.py`) live once at the package root and are imported by all three designs. `src/letterboxd/` mirrors this layout module for module (its own `pseudo_users.py`, `features.py`, `movie_features.py`, `build_rows.py`, `train_xgboost.py`, `train_neural.py`, `analyze.py`, `web_export.py`), sharing no code, data, or model artifacts with Rotten Tomatoes – the gsimonx37 CSVs are the one shared external download, read independently by each project’s own join. `tests/` covers the formula, the random-holdout and paired-episode protocols, partition disjointness, and the feature contract for both projects (`test_design2_features.py`, `test_letterboxd_features.py`) – the genre count’s leave-target-out property, the theme kernel’s weighting and masking, the actor rankings’ tie-breaks and padding, and the count-is-the-mask invariant across every affinity block.

Reproducibility. Python, pandas, scipy.sparse, NumPy, scikit-learn (XGBoost is used through its scikit-learn estimator API; K-means and standardization come from scikit-learn directly), sentence-transformers (the one offline, one-time theme-embedding step – not scikit-learn, since scikit-learn has no sentence-embedding model), PyTorch, and matplotlib. The pseudo-user substrate and feature code are shared, seeded modules (see Repository layout above), so the cross-design comparison is exact. Both projects run entirely client-side in the browser (Vite/React/TypeScript, `web/`), hosted as a static GitHub Pages site with no server. Run `make rt` and `make lb` from the repository root (each stage is also its own target, e.g. `make rt-xgboost`); all random seeds are fixed.